실전 1 보고서 2부 ⑥ Holdout 성능

> - **과제**: 금융사 사내 문서에서 개인정보 7종을 찾는 탐지기를 규칙 기반·토큰 분류·LLM 세 방식으로 비교하고, 운영에 쓸 탐지기를 근거 있게 고른다.
> - **데이터**: 제공 `pii_synth_finance` dev 100건(정답 548개) · test 200건(정답 1,165개) / 추가 시험 데이터: 엣지 케이스 64건(정답 116개), 일상 문서 1,983건(정답 3,375개) / **멘토 holdout 187건(정답 912개)**
> - **채점**: 멘토 제공 `run_compare.py` (strict·partial 정밀도/재현율/F1, mask_rec, 라벨별·어려운 사례 재현율, 일관성)
> - **작성**: 2026-10-26 · 저장소 `hosung` 브랜치 `실전프로젝트1/` · 작업 기록 전체는 `worklog.md` (21장)

보고서 구성: 0 요약 및 결론 · **1부** ① 데이터 관찰 리포트 · ② 방식별 비교표 · ③ 오류 유형 분석 · ④ 탐지기 선정 근거 · **2부** ⑤ Holdout 데이터 관찰 · ⑥ Holdout 성능 · ⑦ Holdout 오류 분석 · ⑧ Holdout 개선 내역 · 부록 개선 과정과 데이터 사용 기록 (`report/` 폴더)

6-1. 측정 조건 — 고치기 전에 먼저 채점

- 탐지기 코드(`detectors/`, `postprocess.py`, `make_union.py`)는 holdout을 받기 전 마지막 커밋(`4c4aa0b`, 규칙 v7) 그대로. `git diff`로 변경 없음을 확인했다. 바꾼 것은 `--split holdout` 경로 추가뿐이다.
- 채점기는 멘토가 함께 준 새 `run_compare.py`로 교체했다. holdout용 어려운 사례 항목만 추가되었고, **교체 후 dev·test 채점 결과가 이전과 완전히 같음**을 확인했다.
- holdout 문서·오류를 보기 전에 채점부터 실행했다. 예측: `preds/holdout_before/`, 채점: `results/holdout_before/`
- 시간: 같은 PC의 CPU, 문서를 하나씩 처리, 모델 로딩 제외.

6-2. 개선 전 성능 — dev·test와 나란히 (최종 탐지기 = 규칙 + 토큰)

데이터	strict P	strict R	strict F1	partial R	mask_rec	ms/문서
dev (100건)	0.987	1.000	0.994	1.000	1.000	—
test (200건)	0.968	0.997	0.982	1.000	1.000	55.1~112.0
일상 문서 평가용 (994건, 팀 생성)	0.909	0.979	0.942	0.991	0.988	83.2
멘토 holdout (187건)	**0.912**	**0.978**	**0.944**	**0.993**	**0.993**	**48.1**

(dev·test·일상 문서도 holdout과 같은 v7 기준. test 시간은 같은 날 오전·오후 측정이 PC 부하로 약 2배 달라 범위로 적었다. 일상 문서는 평균 78자로 짧아 문서당 시간이 다르다.)

구성 요소별 (holdout, 개선 전)

방식	strict P	strict R	strict F1	partial R	mask_rec	ms/문서
규칙 기반	0.936	0.910	0.923	0.919	0.961	3.7
토큰 (atonlee)	0.931	0.950	0.940	0.980	0.979	44.4
규칙 + 토큰	0.912	**0.978**	**0.944**	**0.993**	**0.993**	48.1

- 1만 건 환산: 규칙 + 토큰 약 8분(이번 측정), PC 부하가 클 때 약 2배
- 일관성: 규칙·토큰 모두 결정적이라 같은 문서는 언제나 같은 결과(100%)

6-3. 라벨별 재현율 (정밀도), 개선 전

라벨	정답 수	규칙	토큰	**규칙 + 토큰**
person	560	0.880 (0.91)	0.993 (0.98)	**0.995** (0.90)
phone_number	147	1.000 (1.00)	0.952 (0.93)	**1.000** (0.94)
email	43	1.000 (1.00)	1.000 (1.00)	**1.000** (1.00)
address	52	0.962 (1.00)	1.000 (0.96)	0.962 (1.00)
주민등록번호	43	1.000 (1.00)	1.000 (0.98)	**1.000** (0.98)
account_number	41	0.878 (1.00)	0.829 (0.83)	0.976 (0.95)
card_number	26	1.000 (1.00)	1.000 (0.93)	**1.000** (1.00)

(partial 기준. 합집합 주소 재현율이 토큰 단독보다 낮은 것은 위치가 겹칠 때 규칙 쪽 경계를 쓰는데, 규칙이 주소 앞 우편번호를 빼고 잡았기 때문 → ⑦ 7-3)

6-4. 어려운 사례 재현율, 개선 전

항목	개수	규칙	토큰	**규칙 + 토큰**
조사 붙은 이름	157	0.854	0.994	**1.000**
띄어 쓴 이름	205	0.883	0.990	0.995
이름만 쓴 경우	135	0.763	0.985	0.985
동명이인	102	0.951	1.000	**1.000**
전화 형식 변형	89	1.000	0.944	**1.000**
카드 형식 변형 (새)	25	1.000	1.000	**1.000**
주민번호 형식 변형 (새)	30	1.000	1.000	**1.000**
계좌 형식 변형 (새)	23	0.783	0.696	0.957
주소 형식 변형 (새)	41	0.951	1.000	0.951
외자 이름 (새)	38	**0.289**	0.921	0.947
복성 (새)	25	0.920	1.000	**1.000**

미끼 오탐 (개선 전): 대표번호 미끼 문서 8 · 이름 같은 단어 8 · 개인정보 없는 문서 8 (모두 규칙 쪽에서 나옴, 토큰은 0)

6-5. 해석

- **최종 탐지기는 처음 보는 데이터에서도 놓치지 않음이 유지됐다**: mask_rec test 1.000 → holdout 0.993, 정답 912개 중 미탐 3개. 팀 생성 일상 문서(0.988)와 비슷한 수준이라, 1부에서 "처음 보는 문서 성능의 대표값"으로 쓴 일상 문서 점수가 holdout을 잘 예측했다.
- **규칙 단독은 크게 떨어졌다**: mask_rec 0.997(test) → 0.961, 이름 재현율 0.880, 외자 이름 0.289. dev·test 서식에 맞춘 규칙의 한계가 드러났고, **토큰 모델과 합치기로 한 1부 결정④이 맞았다는 근거**다.
- **토큰 단독은 계좌에 약했다**: 계좌 재현율 0.829, 계좌 형식 변형 0.696 → 규칙이 메움.
- **정밀도가 가장 많이 떨어졌다**: test 0.968 → 0.912. 하한선 0.90은 넘지만 여유가 거의 없다. 원인은 ⑦에서 분석했다.
- 개선 후 점수는 ⑧ Holdout 개선 내역에 따로 적었다(holdout에 맞춘 값이라 일반화 성능 아님).